

Universidad Nacional de Loja

FACULTAD DE LA ENERGÍA, LAS INDUSTRIAS Y LOS RECURSOS NATURALES NO RENOVABLES

CARRERA DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

TERCER CICLO "A"

ASIGNATURA: Estructura de Ordenadores

ELABORADO POR:

Lucas Sebastian Orellana Martinez

Docente encargado: PhD (c). Cristian R. Narváez Guillén

Unidad: 1era Unidad

LOJA – ECUADOR

Introducción a la Persistencia y Jerarquía de Memoria

En el ámbito de las ciencias computacionales, la persistencia de datos es la capacidad de un sistema para conservar la información más allá de la ejecución de un proceso o del reinicio del hardware. Para lograr esto, el software debe interactuar de forma continua con la jerarquía de memoria del computador, la cual se divide principalmente en memoria volátil y memoria no volátil. Mientras que la RAM ofrece tiempos de acceso extremadamente rápidos ideales para estructuras de datos dinámicas, pierde su información sin energía. Por otro lado, el almacenamiento secundario retiene los datos de forma permanente, pero sus tiempos de acceso son considerablemente más lentos medidos en milisegundos o microsegundos.

El Impacto de la Latencia de Almacenamiento y el I/O

La latencia de almacenamiento se define como el tiempo que transcurre desde que el sistema operativo envía una solicitud de Entrada/Salida (I/O) al disco hasta que este devuelve los datos solicitados. Cuando un programa necesita leer o escribir datos persistentes, debe realizar llamadas al sistema. Si estas operaciones son síncronas, el procesador experimenta un estado conocido como *I/O Wait*, donde el hilo de ejecución se bloquea por completo a la espera del disco. Como los ciclos de reloj del procesador son millones de veces más rápidos que el disco, un mal manejo de las operaciones de I/O degrada drásticamente la velocidad de respuesta del software, haciendo que el usuario perciba la aplicación como "lenta" o congelada. Esta latencia aumenta significativamente si los datos sufren de fragmentación, obligando al a realizar múltiples saltos de lectura en diferentes sectores físicos para recuperar un solo archivo, multiplicando las peticiones de I/O.

Ejemplos Prácticos de Aplicación

Ejemplo 1: Latencia por operaciones síncronas en localStorage (Front-end) La API localStorage de JavaScript que almacena datos en el lado del cliente es un mecanismo de almacenamiento síncrono. Si en el diseño de nuestro código realizamos múltiples escrituras individuales dentro de un bucle.

Mala práctica (Genera alta latencia):

JavaScript

```
for (let i = 0; i < 500; i++) {
   localStorage.setItem(`clave_${i}`, `valor_${i}`); }
```

Buena práctica (Agrupamiento en RAM): Para evitar el impacto del I/O, la estructura de datos debe procesarse primero en la memoria RAM y luego guardarse en disco mediante una sola operación de persistencia.

JavaScript

```
const datos = {};
for (let i = 0; i < 500; i++)
```

```
  datos[`clave_${i}`] = `valor_${i}`;
}
// Se realiza una sola operación de I/O al disco
localStorage.setItem('datos_agrupados', JSON.stringify(datos));
```

Ejemplo 2: Lectura en Bloques (Buffering) vs. Lectura Secuencial Byte a Byte Imaginemos que nuestro software debe procesar un archivo de texto de 1 GB. Si diseñamos un algoritmo que lea el archivo "byte a byte", obligaremos al sistema operativo a realizar mil millones de llamadas de I/O al disco, colapsando el rendimiento. Al utilizar la técnica de *buffering*, cargamos bloques grandes de datos (por ejemplo, fragmentos de 8 KB o 16 KB) en un *array* en la memoria RAM con una sola petición de disco. Una vez en RAM, la manipulación de esos datos ocurre casi de forma instantánea.

Ejemplo 3: Uso de Árboles B y B+ en el diseño de Bases de Datos En bases de datos relacionales, buscar un registro específico iterando fila por fila en el disco (Table Scan) generaría una latencia insostenible. Para solucionar esto, en estructuras de datos se implementan los Árboles B/B+ como índices. Los nodos de estos árboles están diseñados estratégicamente para coincidir con el tamaño de los bloques de paginación del disco (generalmente 4 KB u 8 KB). Gracias a esto, un sistema gestor de bases de datos puede encontrar un registro entre millones haciendo apenas 3 o 4 accesos físicos de I/O (correspondientes a la altura del árbol), en lugar de leer todo el disco.

Conclusiones

En conclusión el rendimiento de una aplicación exige optimizar tanto la RAM como el acceso al disco. Como futuros profesionales, debemos minimizar la latencia y el I/O mediante el agrupamiento de datos, uso de buffers y árboles eficientes para construir software escalable, rápido y fluido.

Bibliografía

- **Maxwell Newage (2021).** ***Persistencia de datos en Javascript: LocalStorage*****. Dev.to. Recuperado de: https://dev.to/maxwellnewage/persistencia-de-datos-en-javascript-localstorage-419mOpenDSA**
- **FreeCodeCamp (s.f.).** ***Cómo usar localStorage en JavaScript*****. Recuperado de: https://www.freecodecamp.org/espanol/news/como-usar-localstorage-en-javascript/**
- **Stack Overflow (2009).** ***How to store persistent data client-side?*****. Recuperado de: https://stackoverflow.com/questions/714420/how-to-store-persistent-data-client-side**